

# 智能体与计算机系统

古金宇

上海交通大学并行与分布式系统研究所

<https://ipads.se.sjtu.edu.cn>

# 球员不打球 vs. 老师不上课



网友：“不是打得不好，是根本不打。”

同学：“古老师啥时候来上课？” “古老师多上点课。”

# CMU/CS:APP

- 第 1 版: 2002
- 第 2 版: 2011
- 第 3 版: 2015

# 古法已死，氛围当立

🗨️ Monica GPT-5.4

“XX已死，XX当立”这种句式，近几年在中文互联网里特别流行，但它的源头其实更早。

## 一、原始出处

它明显化用了中国古代政治口号：

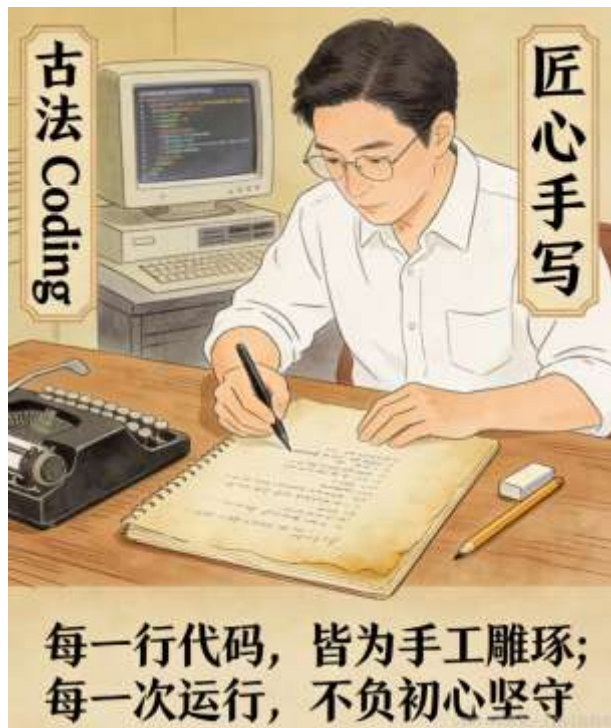
苍天已死，黄天当立  
——东汉末年黄巾起义相关口号

更常见完整版本是：

苍天已死，黄天当立，岁在甲子，天下大吉

这里的“已死 / 当立”本身就是一种宣告旧秩序结束、新秩序登场的强烈表达。

# 古法已死，氛围当立



# 古法已死，氛围当立

Monica GPT-5.4

## 什么是“氛围编程”

氛围编程一般是中文互联网对 **vibe coding** 的翻译或引申说法。

简单说就是：

不太关心底层实现细节，主要靠感觉、靠描述需求、靠 AI 一路生成和修补，把东西先做出来。

# CS:APP

- Computer Systems: A Programmer's Perspective
- **经典内容依然重要**
  - 软件在系统中如何运行的
  - 系统设计的思想依然重要
- **AI时代下内容变化也是必须的**
  - 知识点介绍 =》系统化融合
  - 小轮子 (Hello-World) =》真轮子 (Hello-Agent)

# New CS:APP (理论课)

- ~~Computer Systems: A Programmer's Perspective~~
- Computer Systems: **Agent** Programmer's Perspective
- 本课程依然是一门“系统课”，不是一门“AI 课”
- 选择“智能体 (Agent)”作为主线，因为：
  - 智能体是大家每天都在用的产品 (ChatGPT、Claude、Codex、Cursor)
  - 让智能体稳定工作所需的支撑技术 = 大量计算机系统基础



# MIT: The missing semester (习题课)

## Syllabus

- 1/12/26: [Course Overview + Introduction to the Shell](#)
- 1/13/26: [Command-line Environment](#)
- 1/14/26: [Development Environment and Tools](#)
- 1/15/26: [Debugging and Profiling](#)
- 1/16/26: [Version Control and Git](#)
- 1/20/26: [Packaging and Shipping Code](#)
- 1/21/26: [Agentic Coding](#)
- 1/22/26: [Beyond the Code](#)
- 1/23/26: [Code Quality](#)



ICS Tutorial

# 课程实验数量：0 or 1

- 最多只有1个lab：DIY你的小龙虾 🦞
- 实验评分？
  - Benchmark
  - 答辩

# 讨论

- 你日常使用什么智能体？
- 你认为智能体的工作原理是什么样的？
- 你最担心的智能体技术挑战是什么？



# 智能体

# 来自 Claude 4.7 & GPT-5.4 的定义

给出AI智能体的精简化定义

分享 复制 引用 更多

Monica Claude 4.7 Opus

AI 智能体：能感知环境、自主决策并执行动作以达成目标的 AI 系统。

给出AI智能体的精简化定义

Monica GPT-5.4

精简化定义

AI 智能体：

能感知环境、做出决策并执行动作以完成目标的 AI 系统。

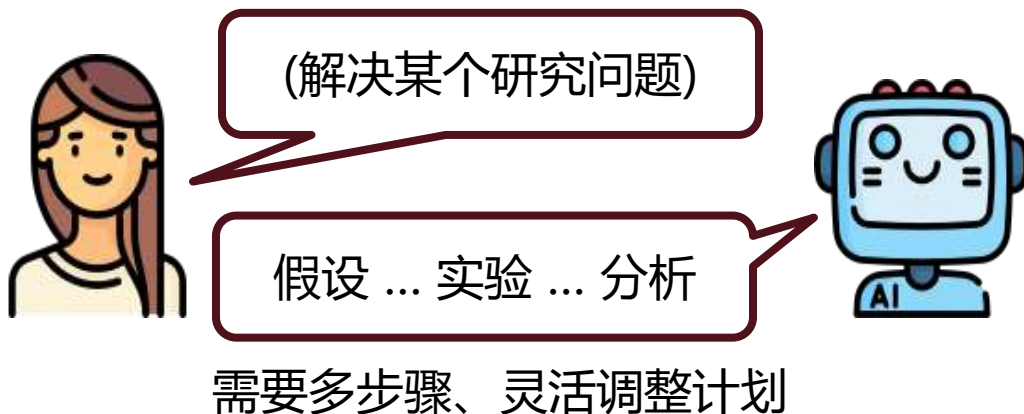
## Chatbot

人类给予明确指令  
AI 一个指令一个动作

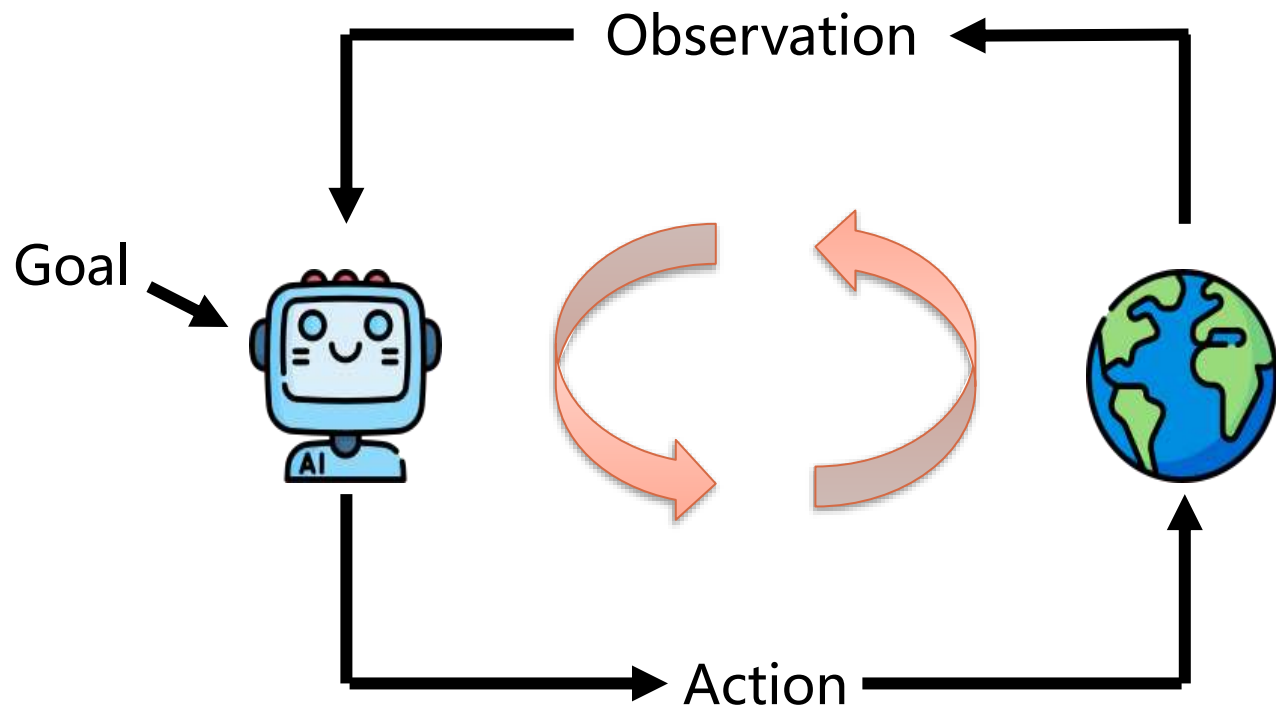


## AI Agent

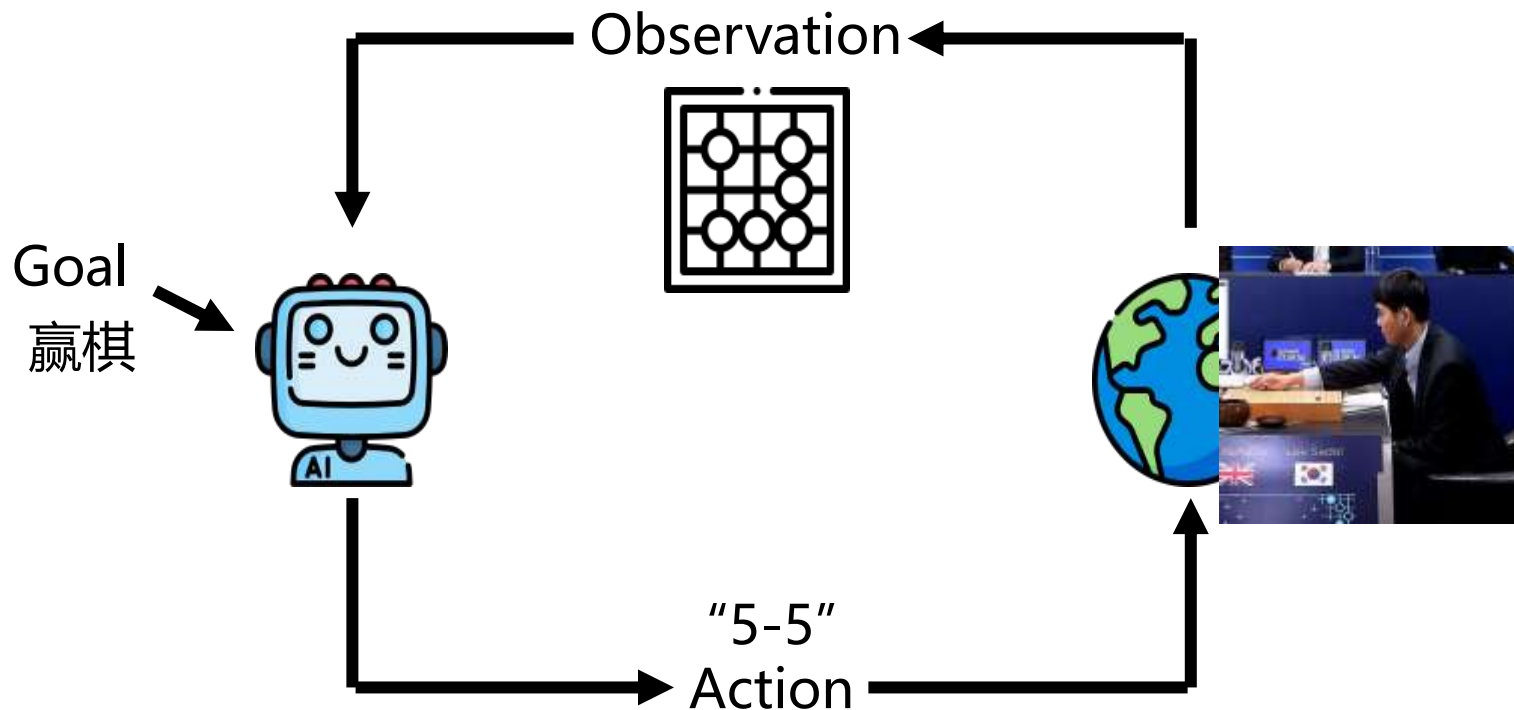
人类给予目标  
AI 自己想办法达成



# AI Agent

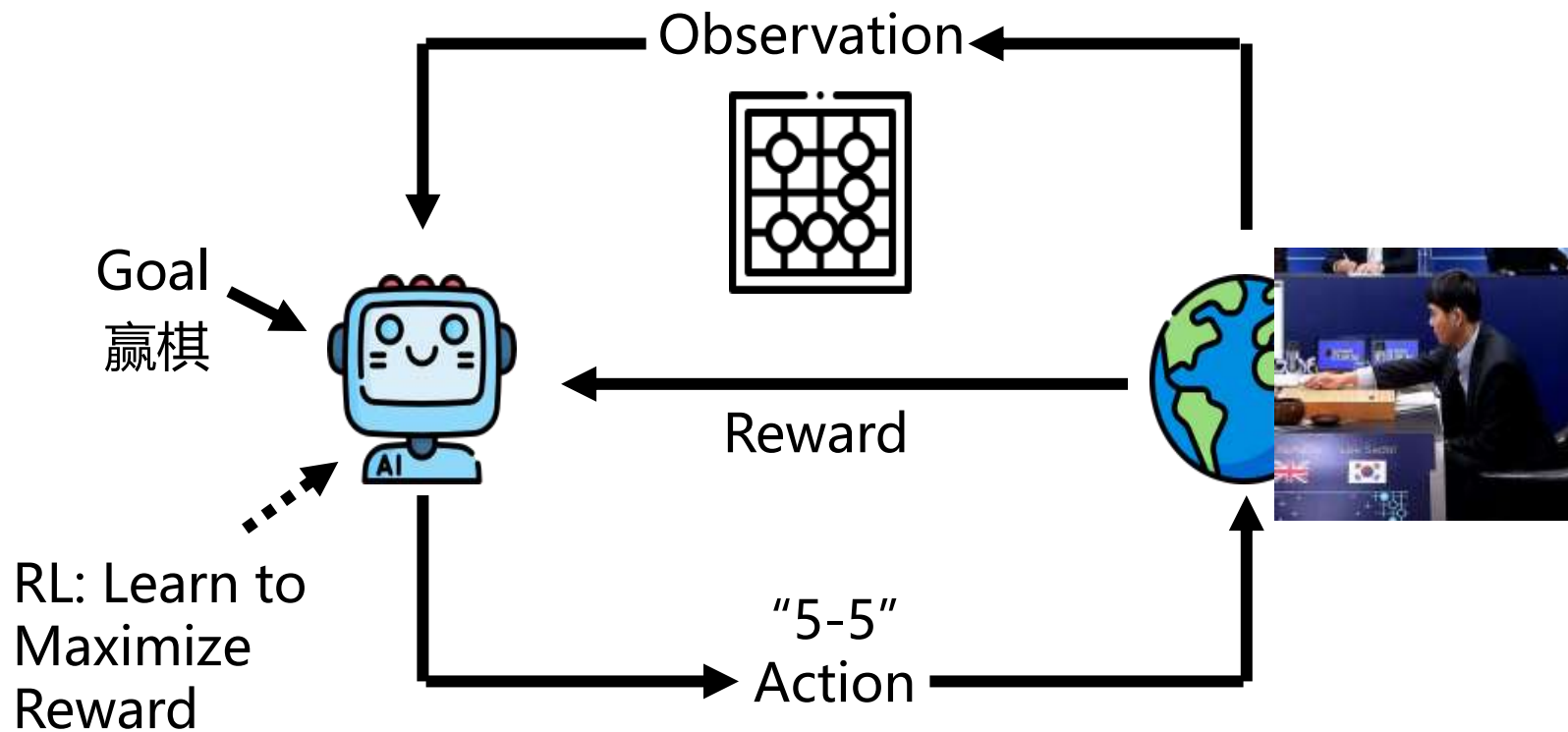


# AI Agent (AlphaGo)



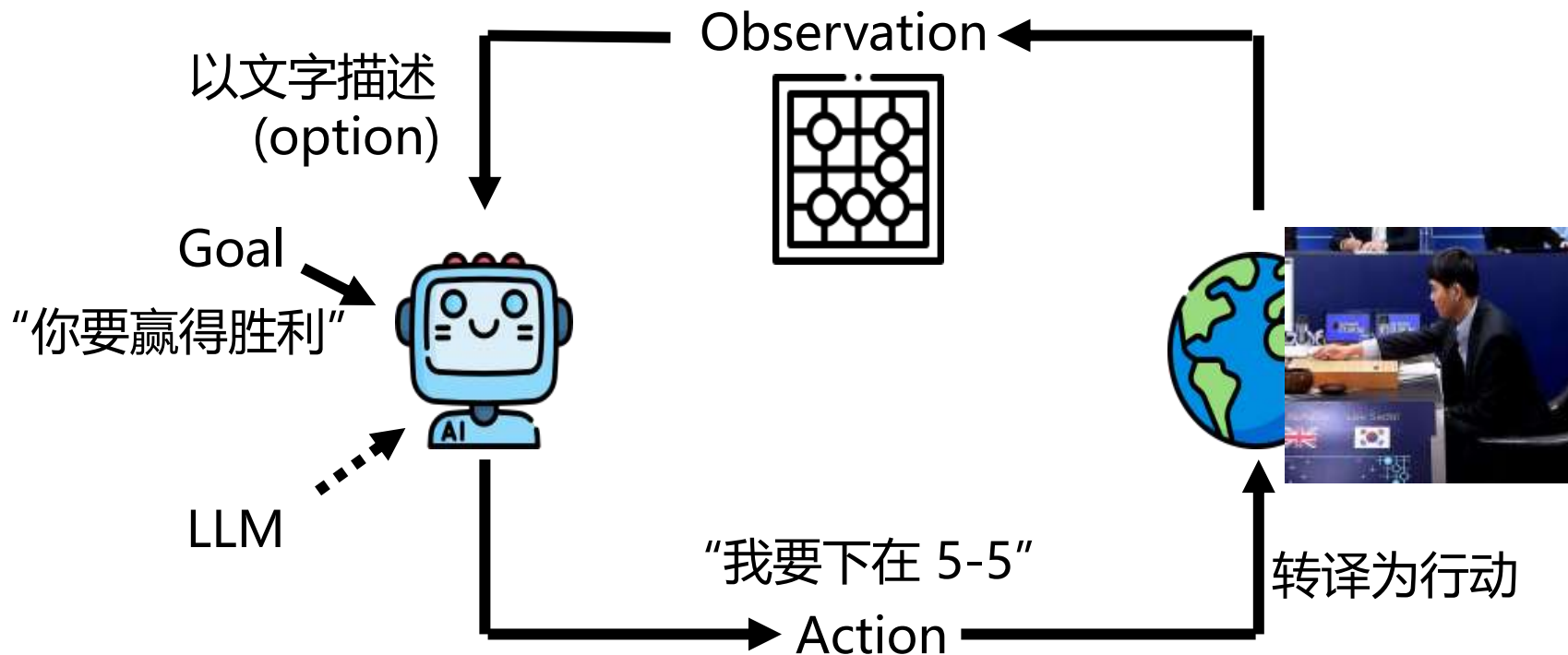


# 如何打造 AI Agent? RL (强化学习) ?



局限：需要为了每一个任务以 RL 训练模型

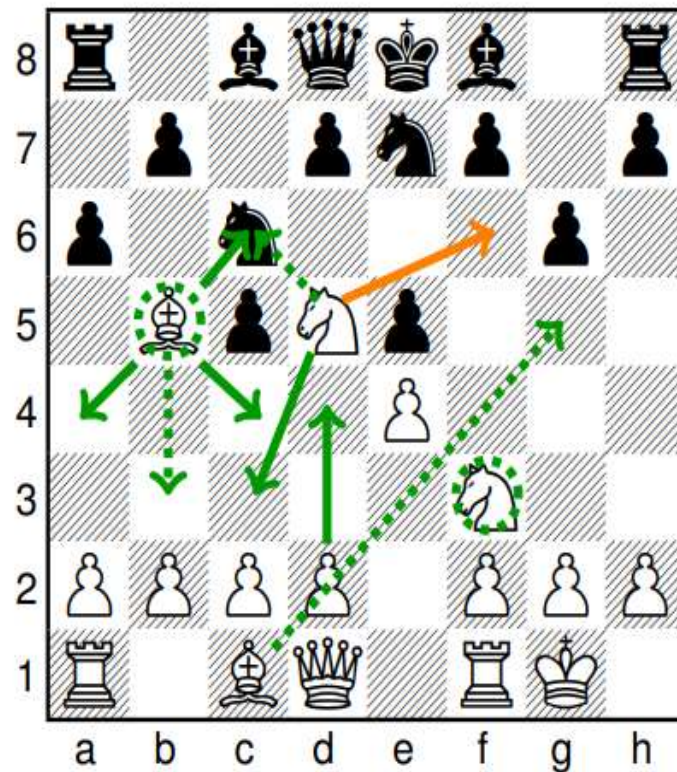
# 如何打造 AI Agent? 直接用 LLM!



# LLM 能不能下棋?

- BIG-bench**

<https://arxiv.org/abs/2206.04615>



In the following chess position, find a checkmate-in-one move.

1. e4 c5 2. Nf3 e5 3. Nc3 Nc6 4. Bb5 Nge7 5. O-O g6 6. Nd5  
a6 7.

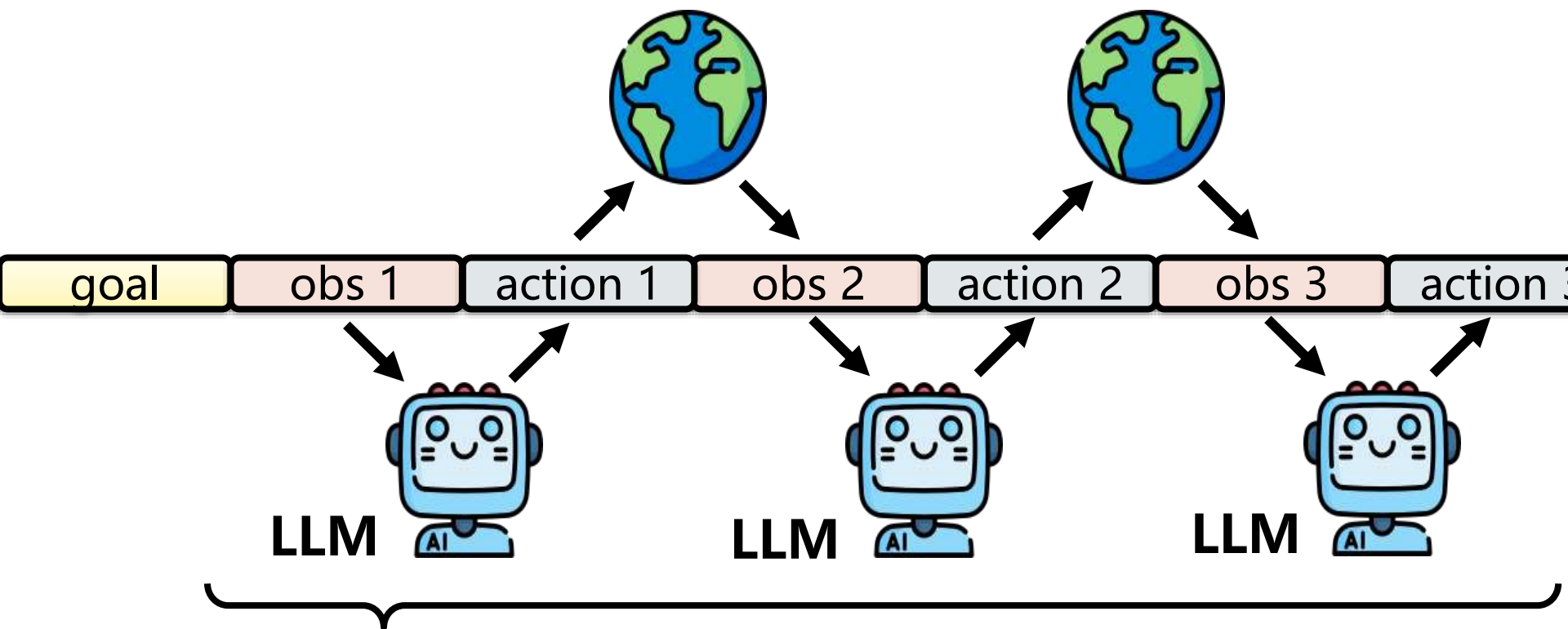
# LLM 能不能下棋?

[https://youtu.be/JHq4EKMg7fI?si=izKsH-GCVnZkooq\\_](https://youtu.be/JHq4EKMg7fI?si=izKsH-GCVnZkooq_)



ChatGPT vs DeepSeek: CRAZY Chess

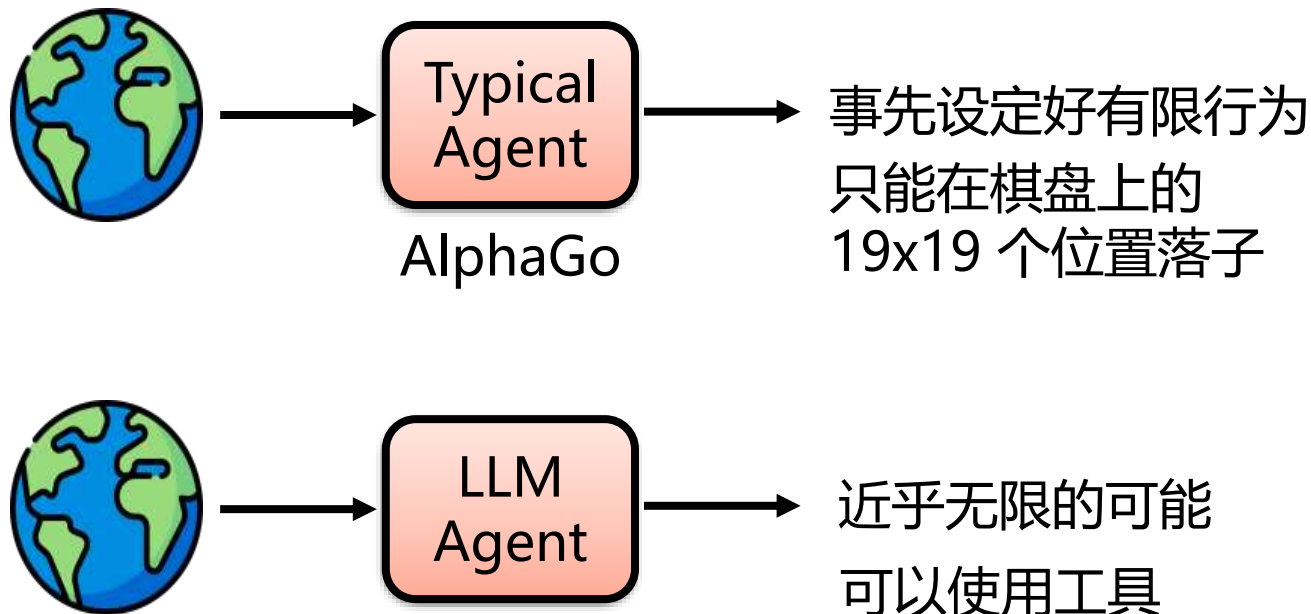
# AI Agent = LLM + Harness



**Agent Loop**

**获取观察+约束执行需要系统工程**

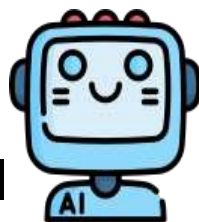
# 以 LLM 运行 AI Agent 的优势



# 以 LLM 运行 AI Agent 的优势

Typical  
Agent

AI  
programmer



Reward = -1

为什么是 -1???

Compile Error



LLM  
Agent

AI  
programmer



更多资讯

Compile Error





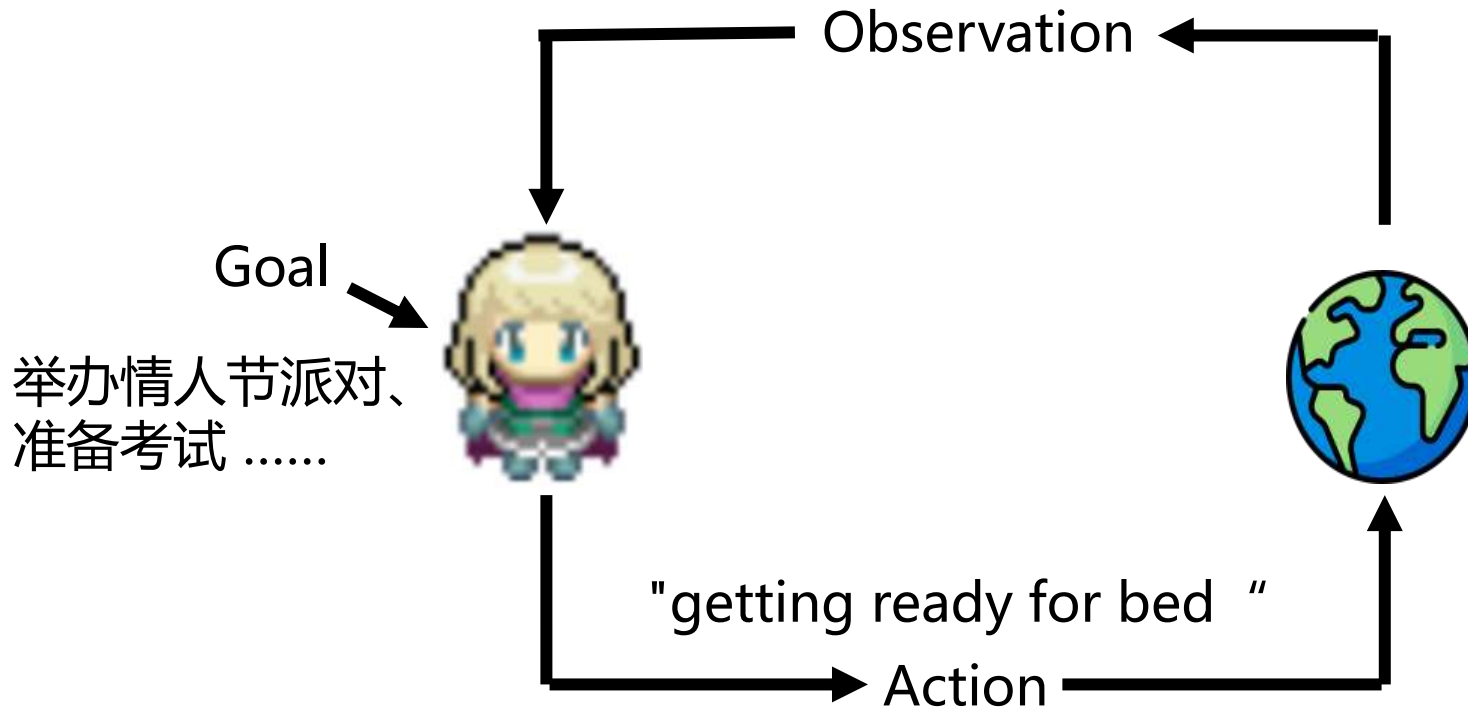
# AI Agent 举例：AI 村民组成的虚拟村庄

<https://arxiv.org/abs/2304.03442>





[node\_749] 2023-02-13 15:33:20: Eddy Lin is studying music theory  
[node\_748] 2023-02-13 15:33:20: cooking area is idle  
[node\_747] 2023-02-13 15:33:20: kitchen sink is idle  
[node\_746] 2023-02-13 15:33:20: behind the cafe counter is idle  
[node\_745] 2023-02-13 15:32:10: Isabella Rodriguez is gathering decorations



# AI Agent 举例: Minecraft 中的 AI NPC



1000 AI NPCs simulate a CIVILIZATION in Minecraft

<https://www.youtube.com/watch?v=2tbaCn0Kp90>

# AI Agent 举例：让 AI 使用手机



这是豆包帮我点的第一单外卖，买了一个椰子

一个椰子没够起送价，他还自动帮我选了一个 1.99 元



下午7:29 · 2025年12月4日 · 16.5万 查看

据其原型机视频演示，豆包通过深度集成安卓系统底层权限，采用类似荣耀Magic OS的“模拟操作”技术，可以直接跨应用调用服务——无需打开美团、淘宝或携程，只需一句话，豆包就能在多个应用之间自动比价、提醒下单、甚至为用户代填地址。



# AI Agent 举例：用 AI 做研究

[Home](#) > [Blog](#) >

## Accelerating scientific breakthroughs with an AI co-scientist

February 19, 2025 · Juraj Gottweis, Google Fellow, and Vivek Natarajan, Research Lead

We introduce AI co-scientist, a multi-agent AI system built as a virtual scientific collaborator to help scientists generate hypotheses and research proposals, and to accelerate the scientific and biomedical discoveries.

## Meet Novix, your AI agentic co-scientist

Accelerate your scientific research from novel idea generation to experimental validation.

[Get Started](#)

# 智能体HARNESS工程的重要能力

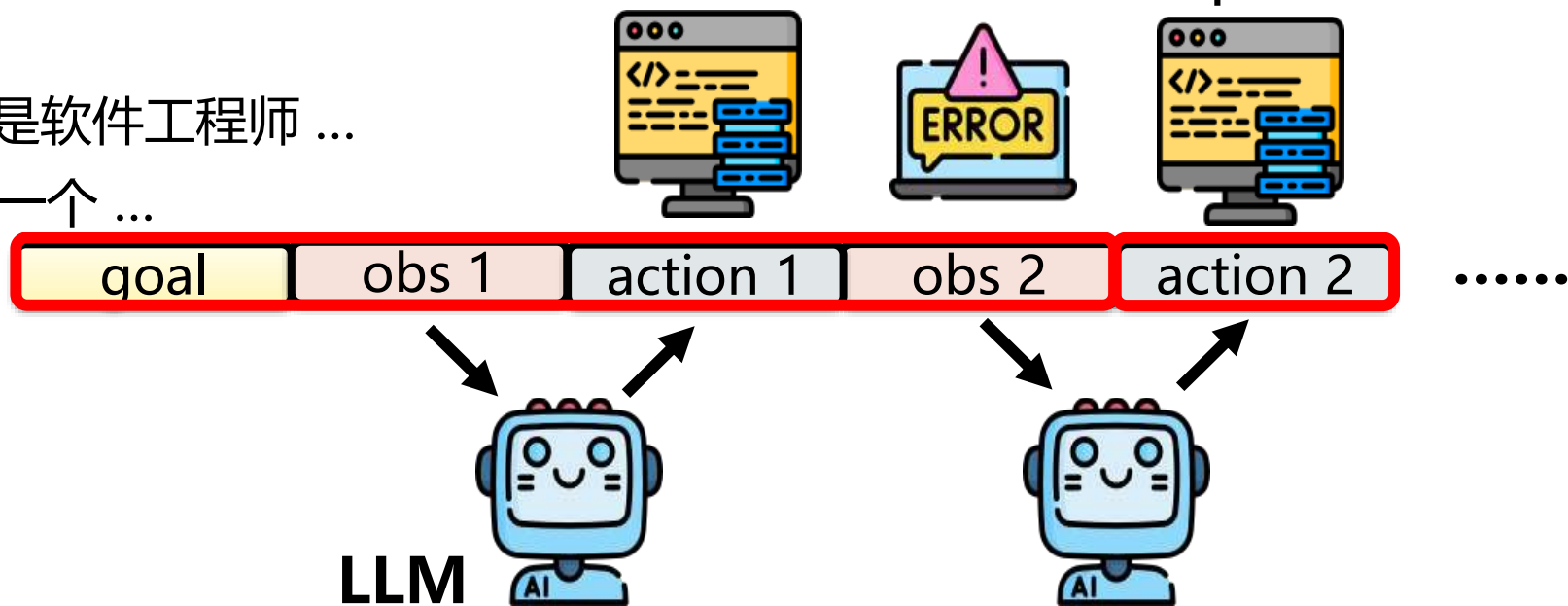
# 根据经验调整行为

智能体记忆

# 根据经验调整行为

你是软件工程师 ...  
写一个 ...

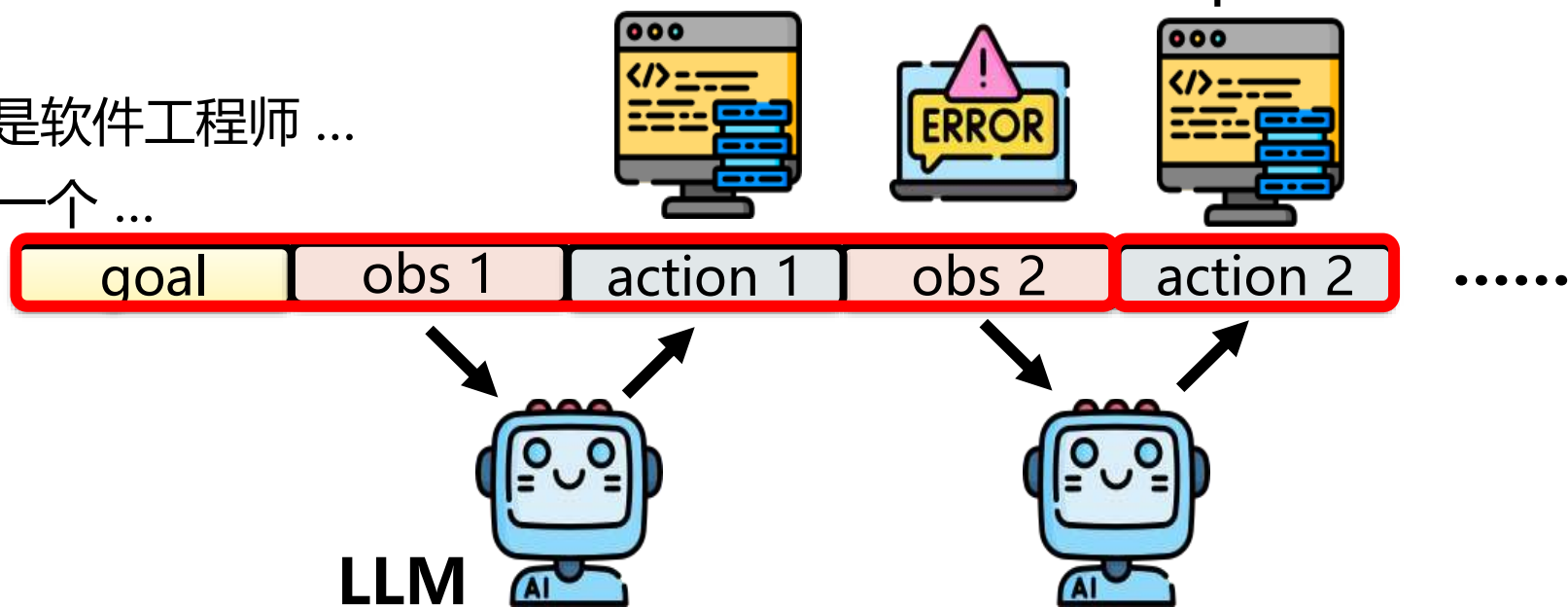
Feedback Update



# 根据经验调整行为

你是软件工程师 ...  
写一个 ...

Feedback Update

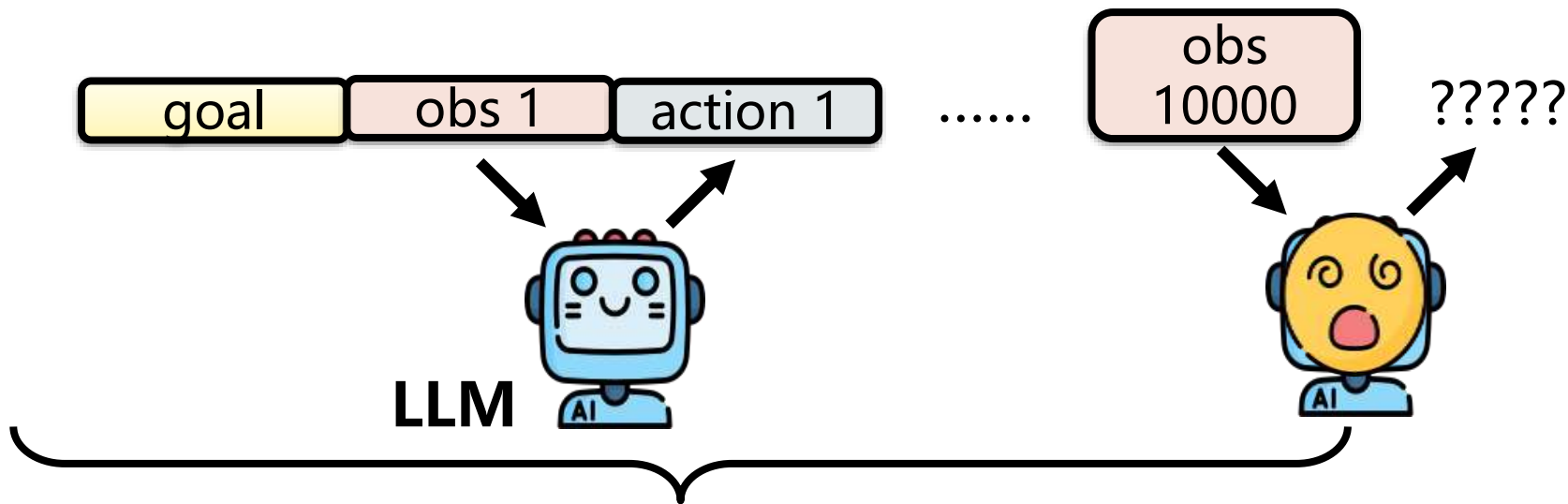


Q: 为什么第二次LLM能把代码写对呢?

提示词工程、上下文工程、Harness工程



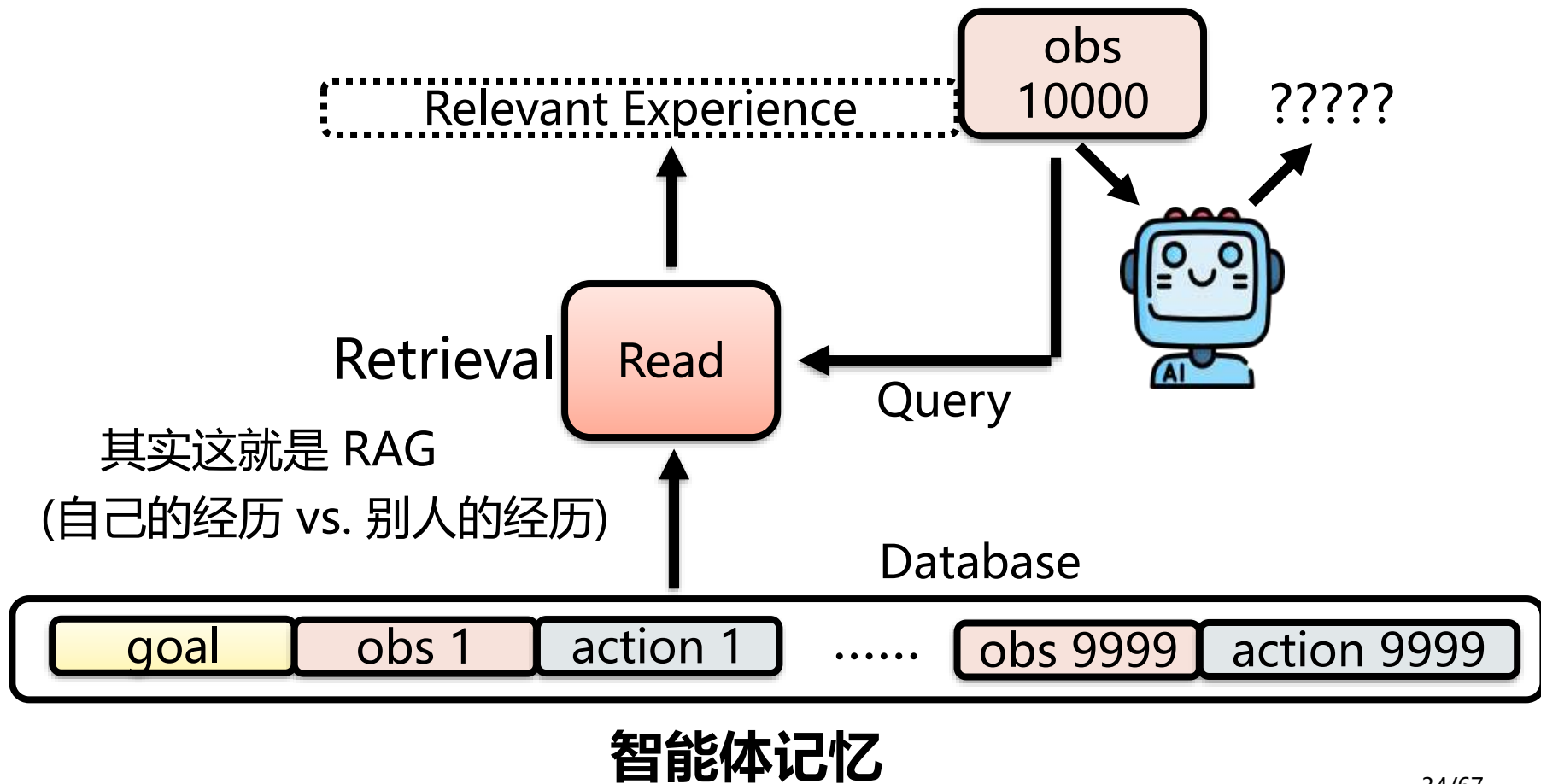
# 根据经验调整行为



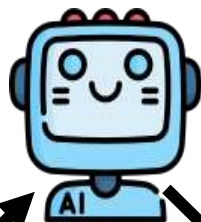
不断回忆整个 Agent 一生的经历 ... ☹

超忆症 (Hyperthymesia)

# 根据经验调整行为



# 根据经验调整行为



Relevant Experience

obs  
10000

action  
10000

obs  
10001

记下来?

goal

obs 1

action 1

.....

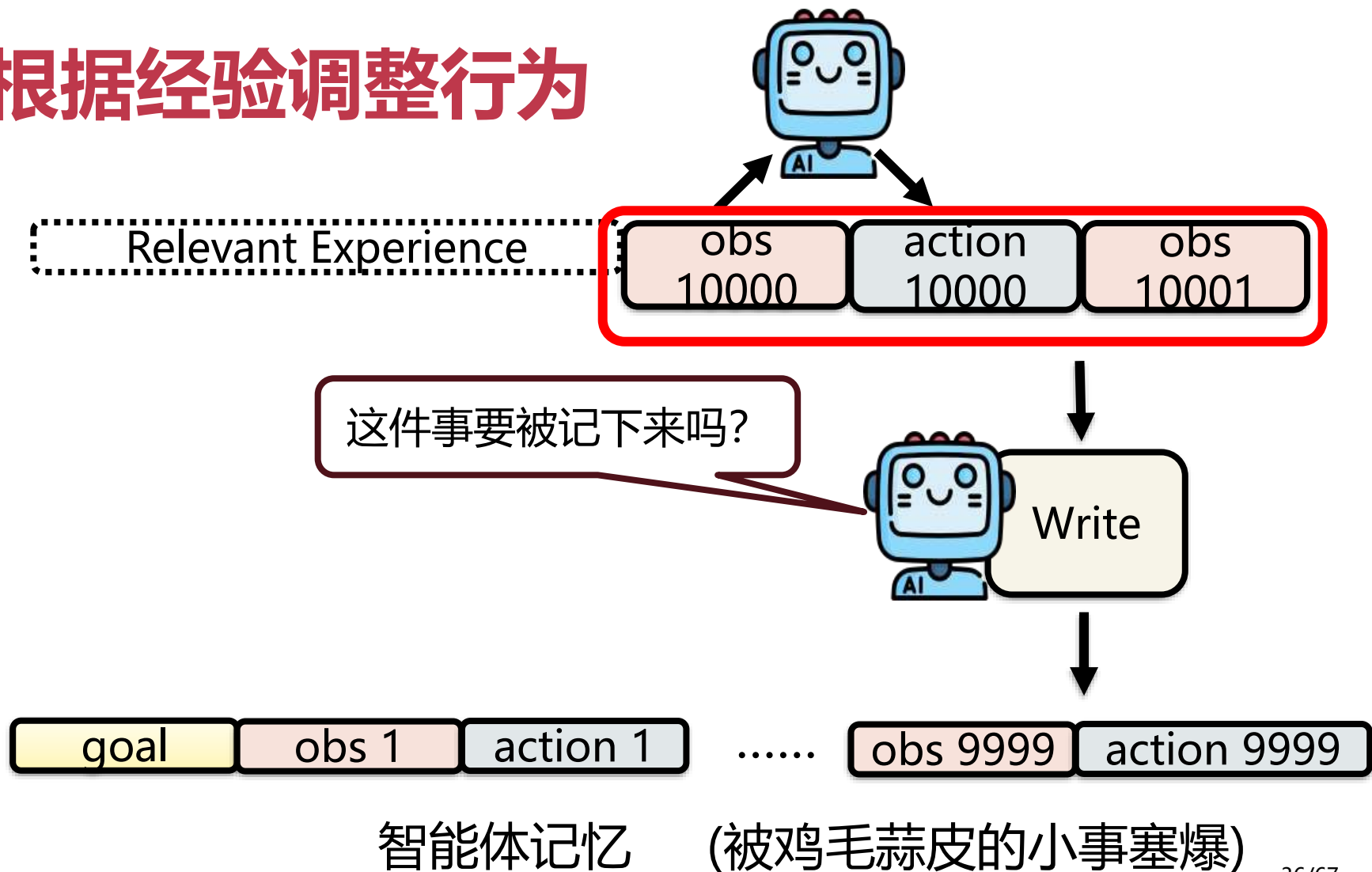
obs 9999

action 9999

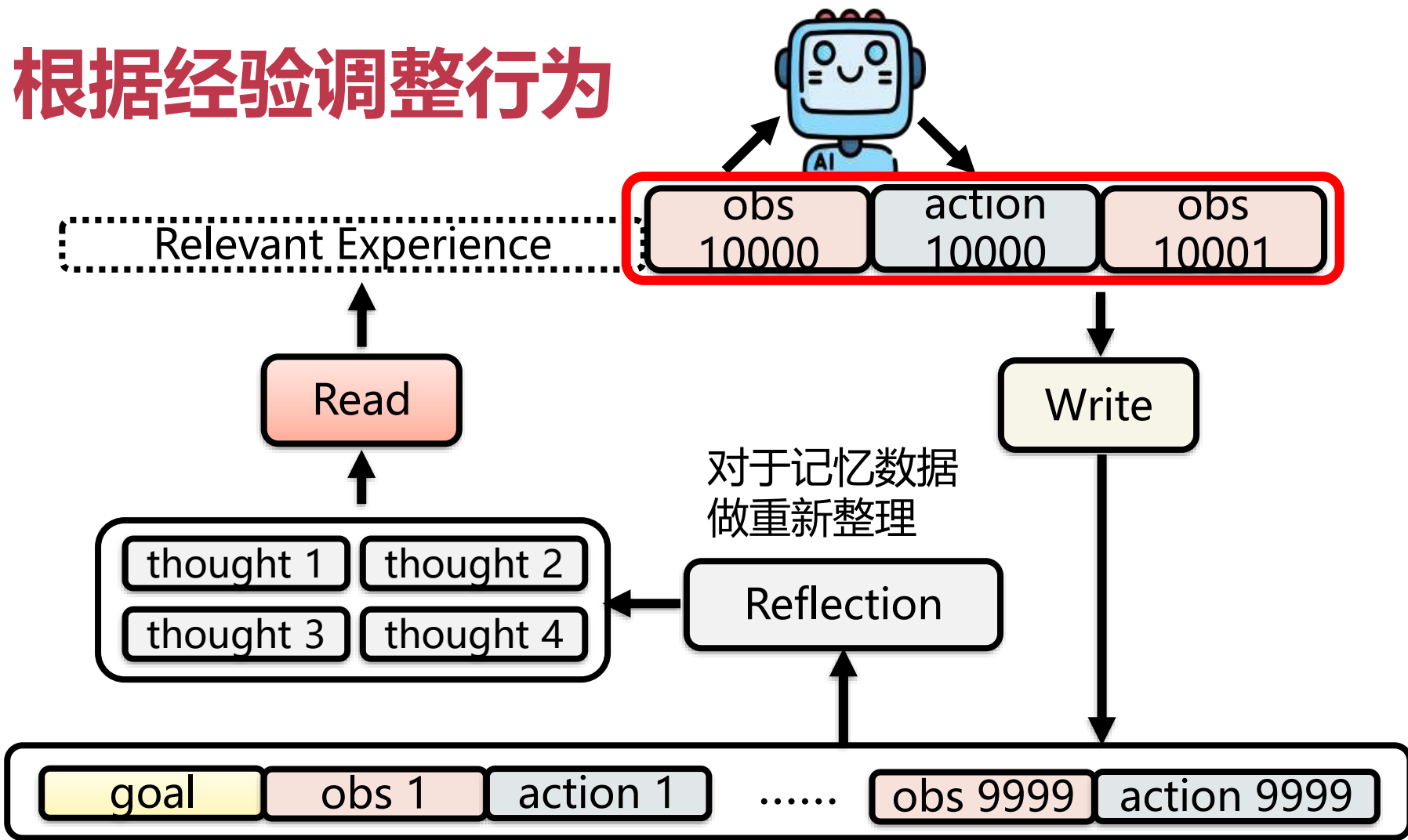
智能体记忆

(被鸡毛蒜皮的小事塞爆)

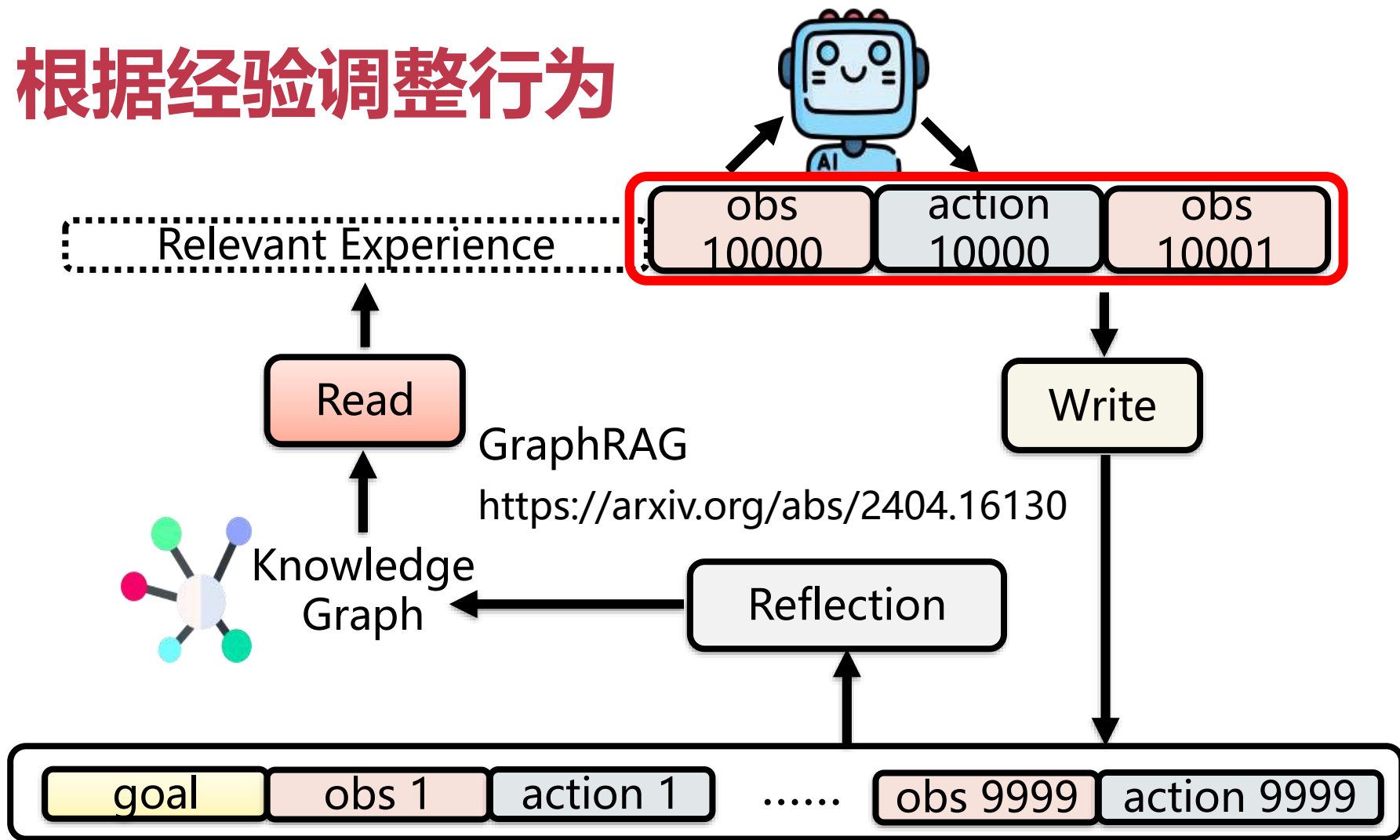
# 根据经验调整行为



# 根据经验调整行为



# 根据经验调整行为



# 智能体记忆与CSAPP

- 层次化存储
- 文件系统
- 虚拟内存

# 上下文越来越长，会发生什么？

- 聊 1 轮： messages = 3 条
  - 聊 10 轮： messages = 30 条左右
  - 聊 100 轮： messages = 300 条左右 ← 开始变慢、变贵
  - 聊 1000 轮： 装不下了
- 
- 核心问题： "东西太多、放不下了，怎么办？ "
  - 这是计算机科学里最古老的问题之一



# "存不下怎么办"的三个朴素方案

- **方案 1: "分层存"**
  - 不是所有东西都放在一起, 按"快但小""慢但大"分成几层
- **方案 2: "按需取"**
  - 要用的时候才加载进来, 只把当前用得着的部分读进来
- **方案 3: "扔旧的"**
  - 旧的、不常用的先踢出去, 给每个信息打"新旧"标签
- **系统思想关键词: 分层 (hierarchy)、按需加载 (on-demand)、替换 (replacement)、压缩 (compression)**

# AI 如何使用工具

---



# 语言模型常用工具

只需要知道怎么使用工具，  
不需要知道其内部运作原理



Search  
Engine



Python



Other AI  
(Different capabilities,  
stronger but costly)

- 工具可以看做是 Function，使用工具就是调用这些 Function
- 使用工具又叫 “Function Call”

(使用工具的方法很多，这边是只是一个通用的方法)

# 如何使用工具

## System Prompt

如果遇到根据你的知识无法回答的问题，使用工具  
把使用工具的指令放在 `<tool>` 和 `</tool>` 中间，使用完工具  
后你会得到输出，放在 `<output>` 和 `</output>` 中间

如何使用  
所有工具

现在你可以使用的工具如下：  
查询某地、某时温度的函式 `Temperature(location, time)`，使用  
范例：`Temperature('台北', '2025.02.22 14:26')`

特定工具  
使用方式

2025 年 3 月 10 日那天下午 2:00，高雄气温如何

## User Prompt

语言  
模型

gpt-4o-mini

这就是一串文字，LLM无法直接执行call

`<tool>Temperature('高雄', '2025.03.10 14:00')</tool>`

(使用工具的方法很多，这边是只是一个通用的方法)

# 如何使用工具

工具使用方式 .....

**System Prompt**

2025 年 3 月 10 日那天下午 2:00，高雄气温如何

**User Prompt**

语言  
模型

gpt-4o-mini

不需要呈现给用户看

`<tool>Temperature('高雄', '2025.03.10 14:00')</tool>`

不需要呈现给用户看

`<output>摄氏 32 度</output>`

Agent 开发者  
先设定好的流程

Temperature

用户看到的输出

(继续去做接龙 .....

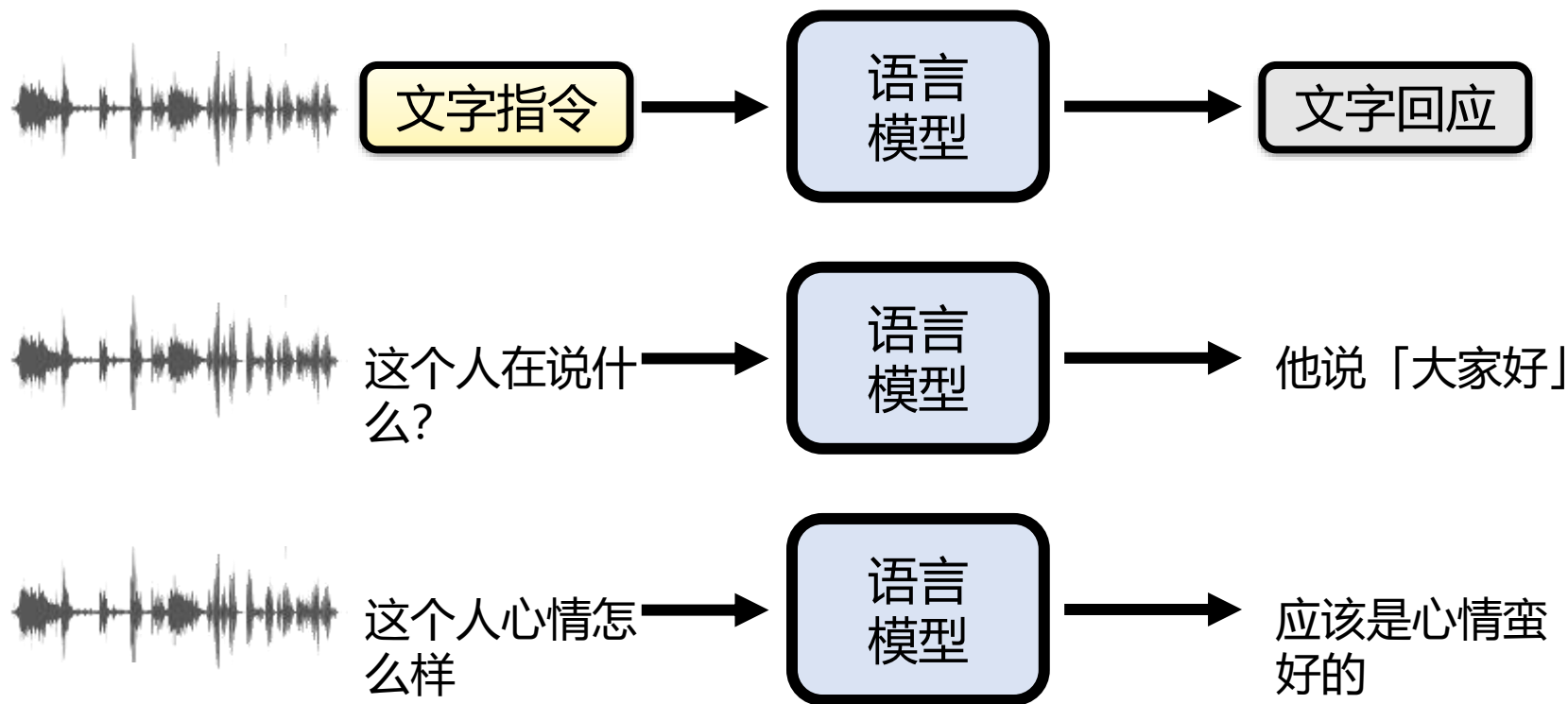
2025 年 3 月 10 日下午 2:00，高雄的气温  
为摄氏32度。

# 最常使用的工具：搜尋引擎

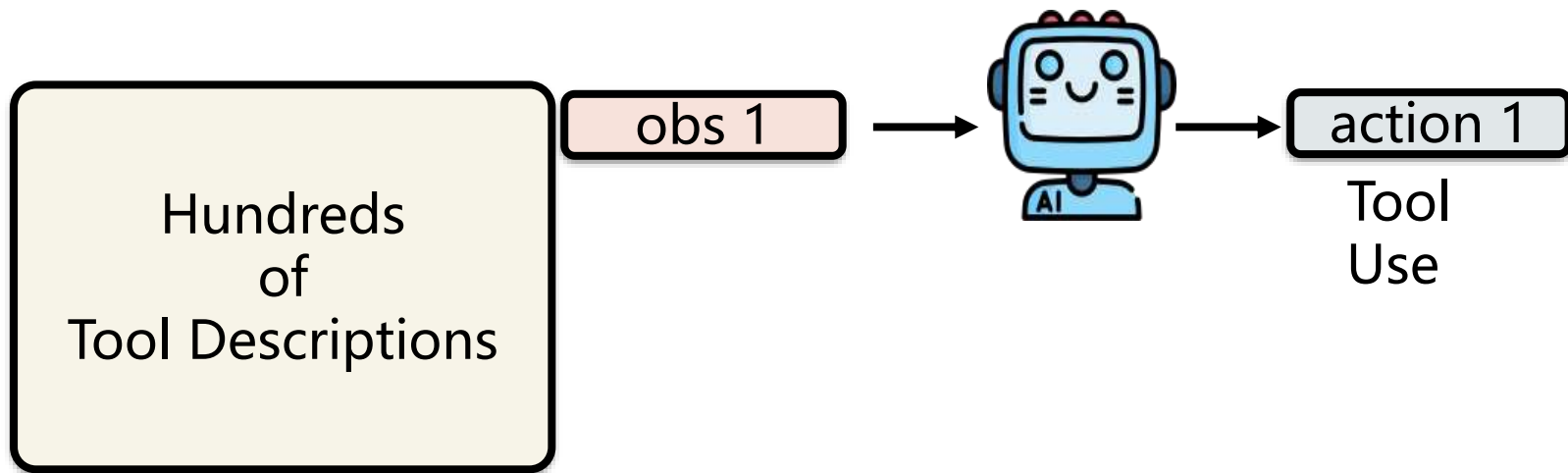
## Retrieval Augmented Generation (RAG)



# 使用其他 AI 作为工具



# 非常多工具怎么办？



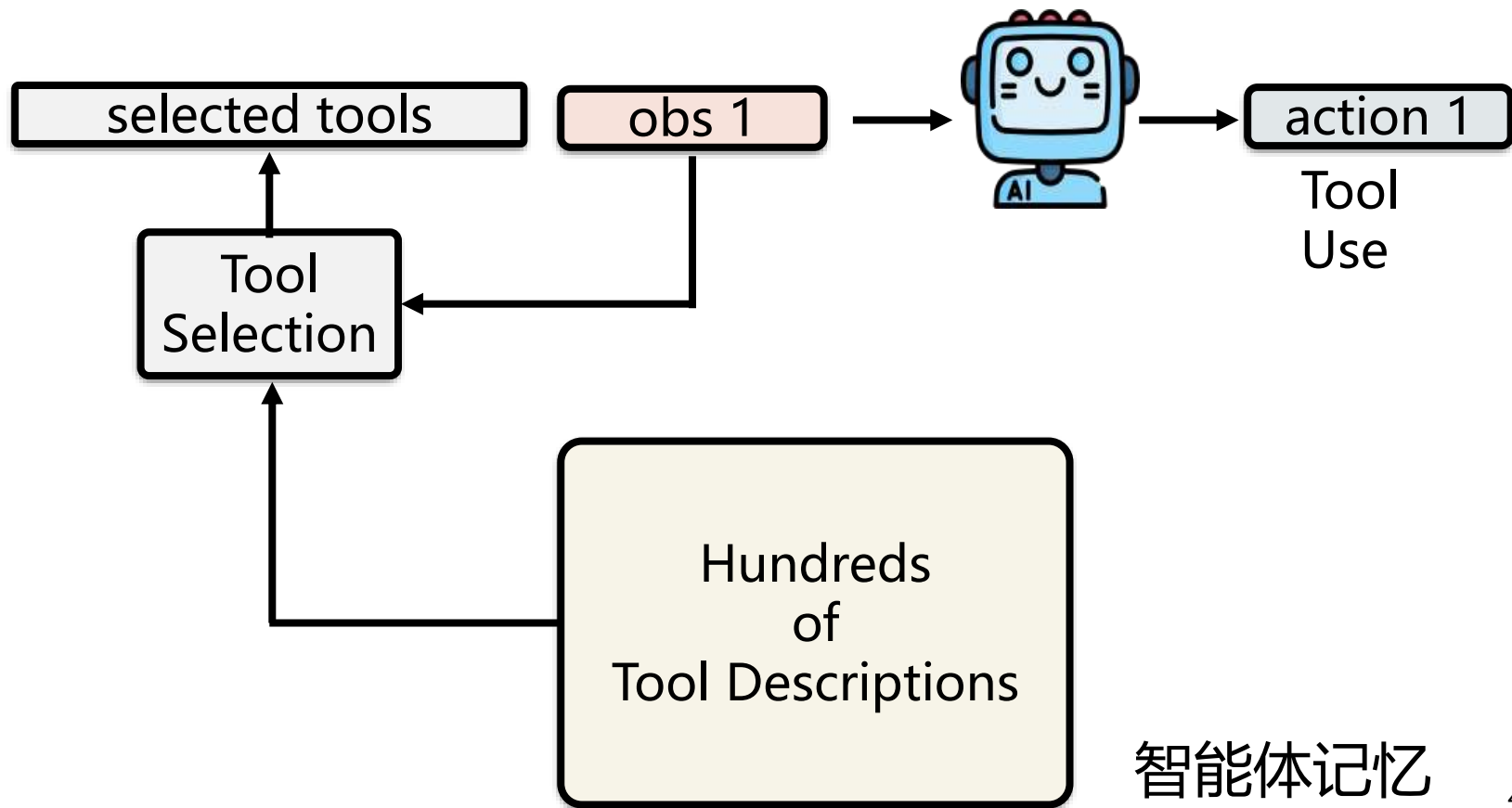
限制1：模型上下文窗口有限

限制2：干扰注意力

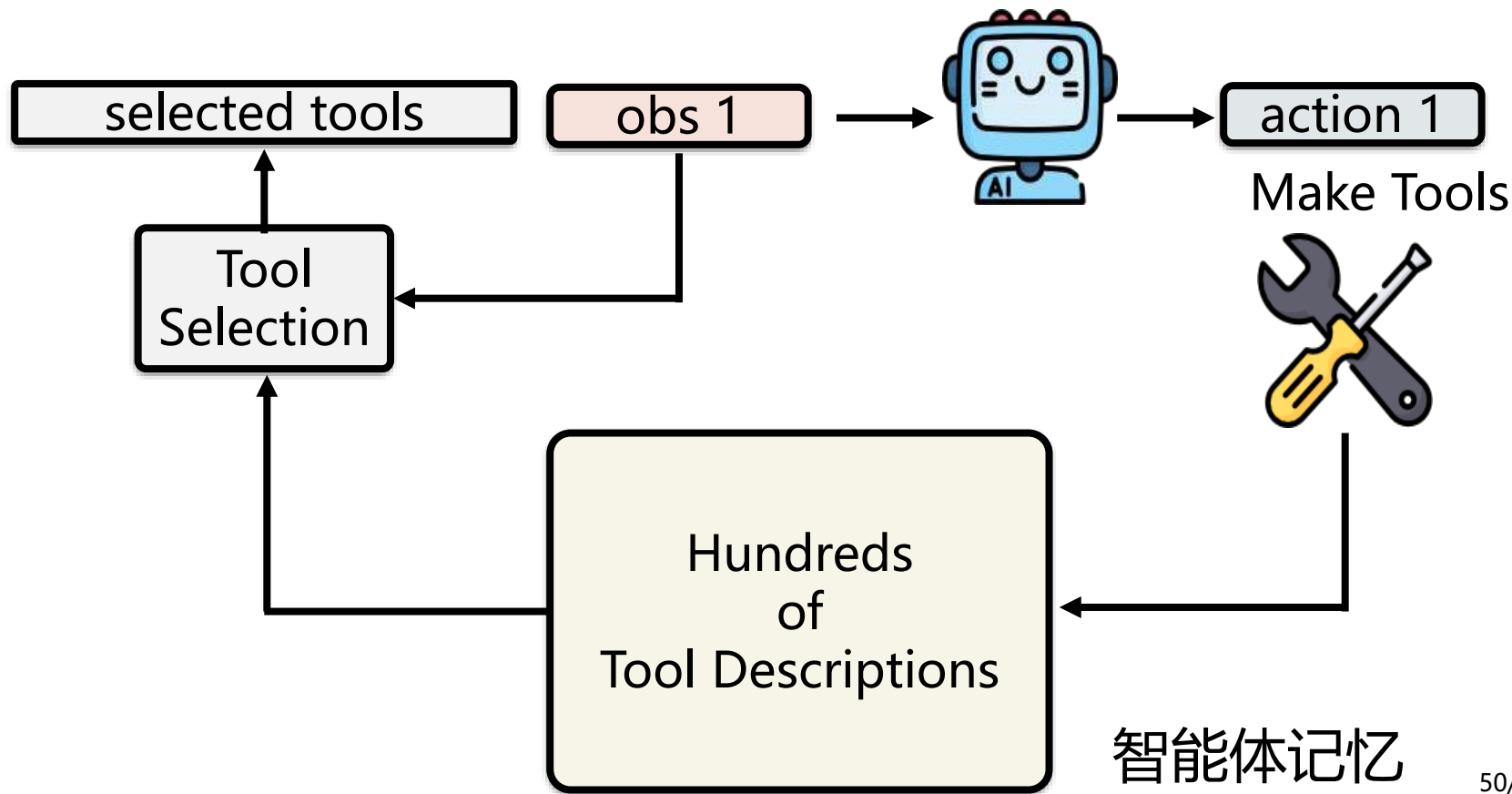


# 非常多工具怎么办？

<https://arxiv.org/abs/2502.11271>

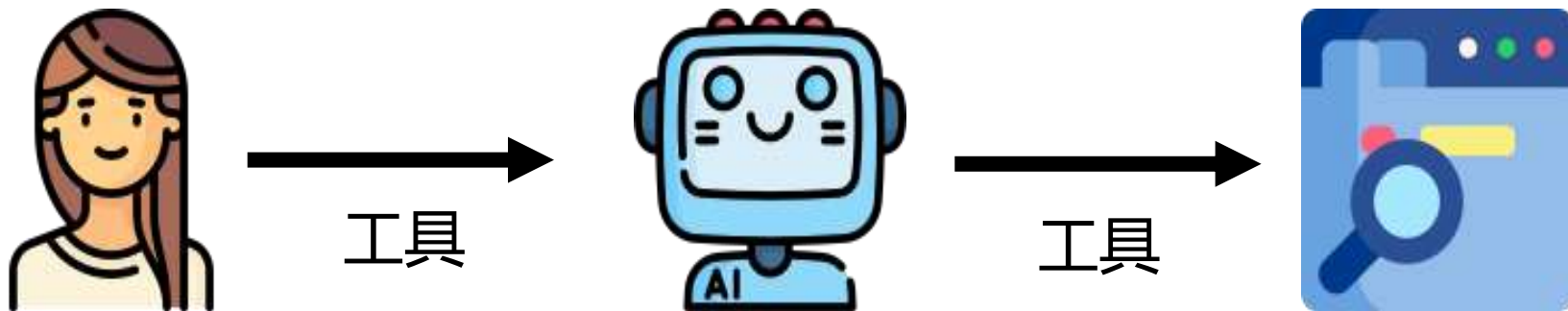


# 模型自己打造工具



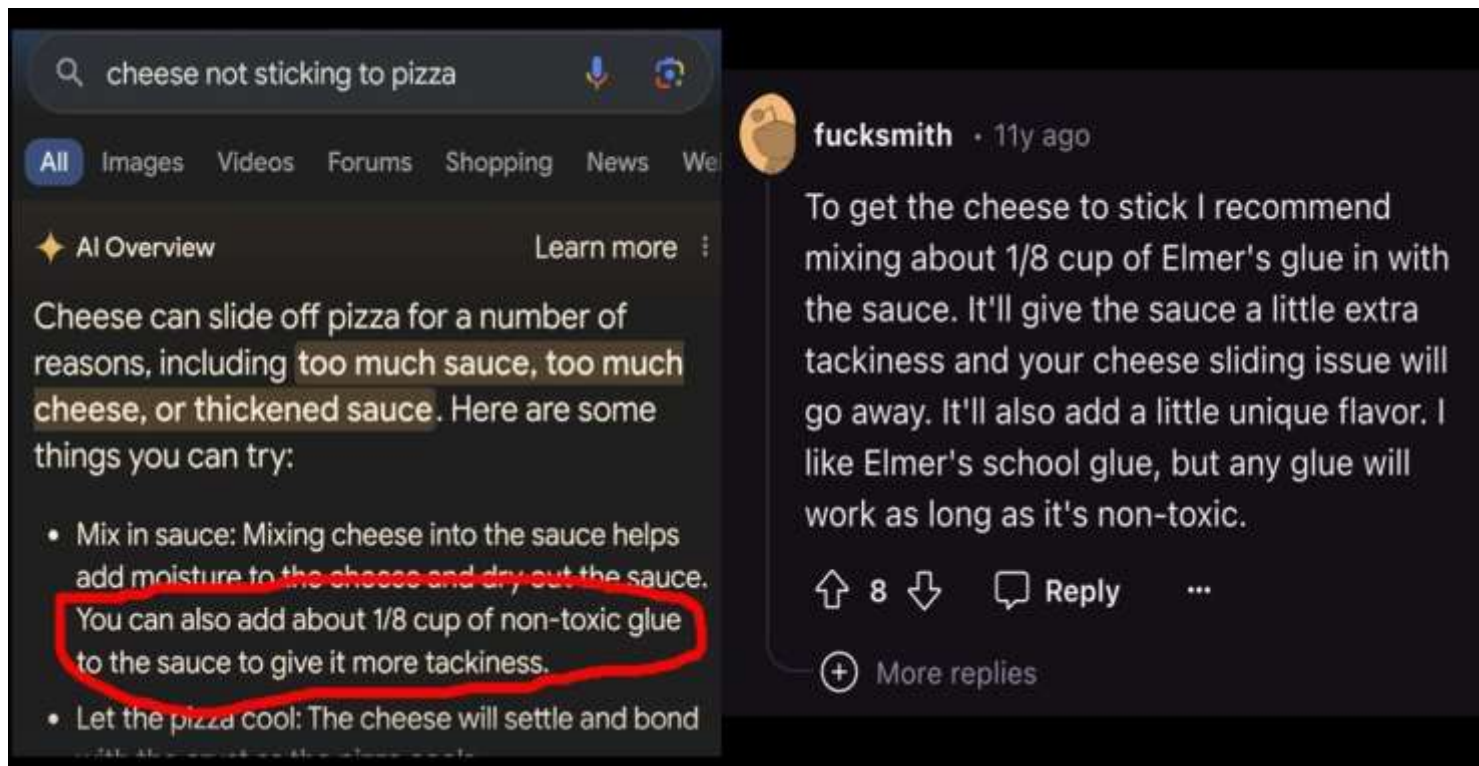
# 可靠性&安全性

因为过度相信工具而犯错 ...



因为过度相信工具而犯错 ...

# 假如工具有问题 ... 以 RAG 为例



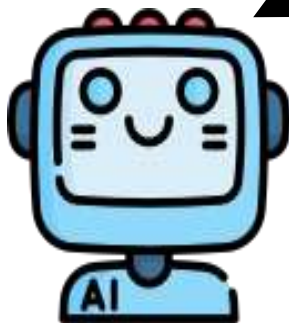
Source of image: [https://www.linkedin.com/posts/petergyang\\_google-ai-overview-suggests-adding-glue-to-activity-7199246664329551872-9VdY/](https://www.linkedin.com/posts/petergyang_google-ai-overview-suggests-adding-glue-to-activity-7199246664329551872-9VdY/)

# 可靠性&安全性

因为过度相信工具而犯错 ...



工具



工具



不要完全相信工具，  
要有自己的判断力



因为过度相信工具而犯错 ...



不要完全相信工具，  
要有自己的判断力

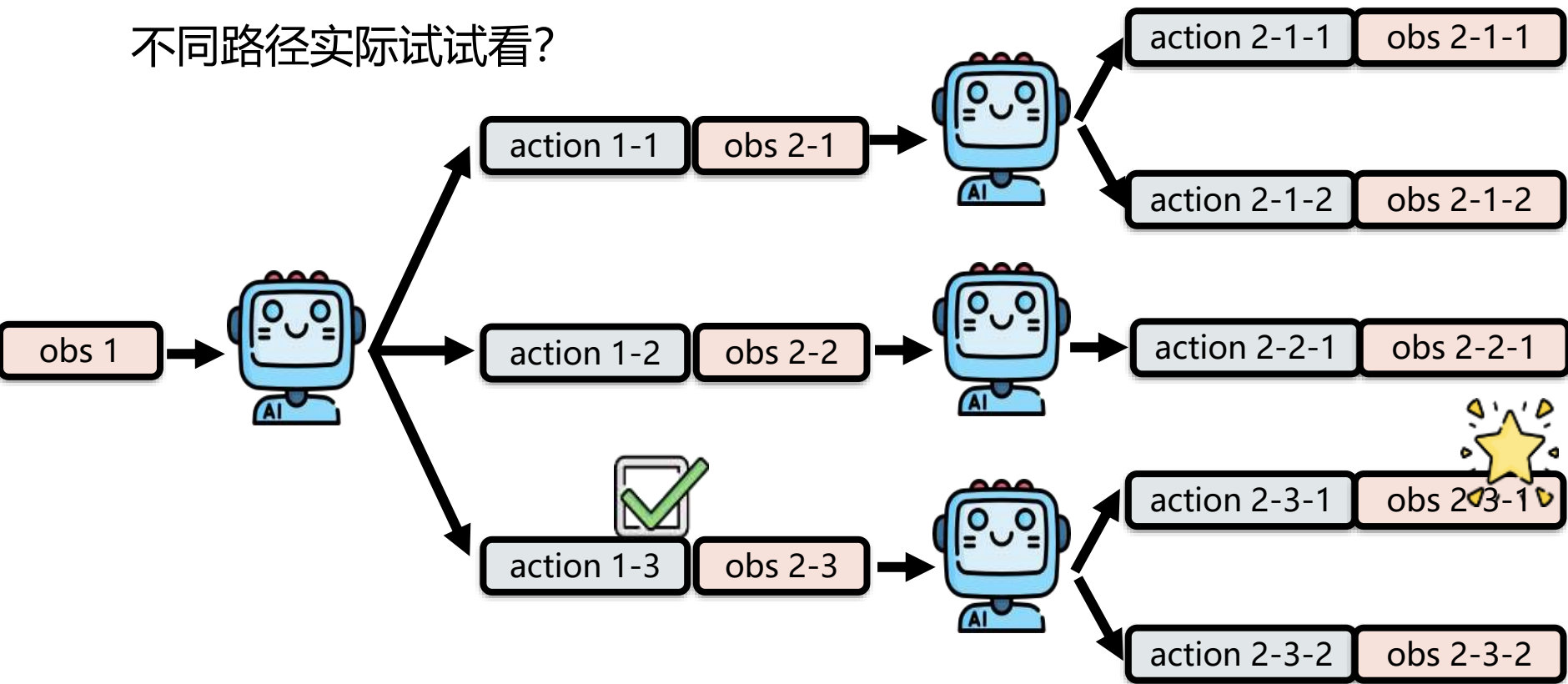
# 智能体工具与CSAPP

- 工具调用协议MCP、CLI (calling convention)
- 启动和运行工具 (进程)
- 调用远程工具 (网络)
- 安全性 (系统权限管控)
- 鲁棒性 (speculative execution)

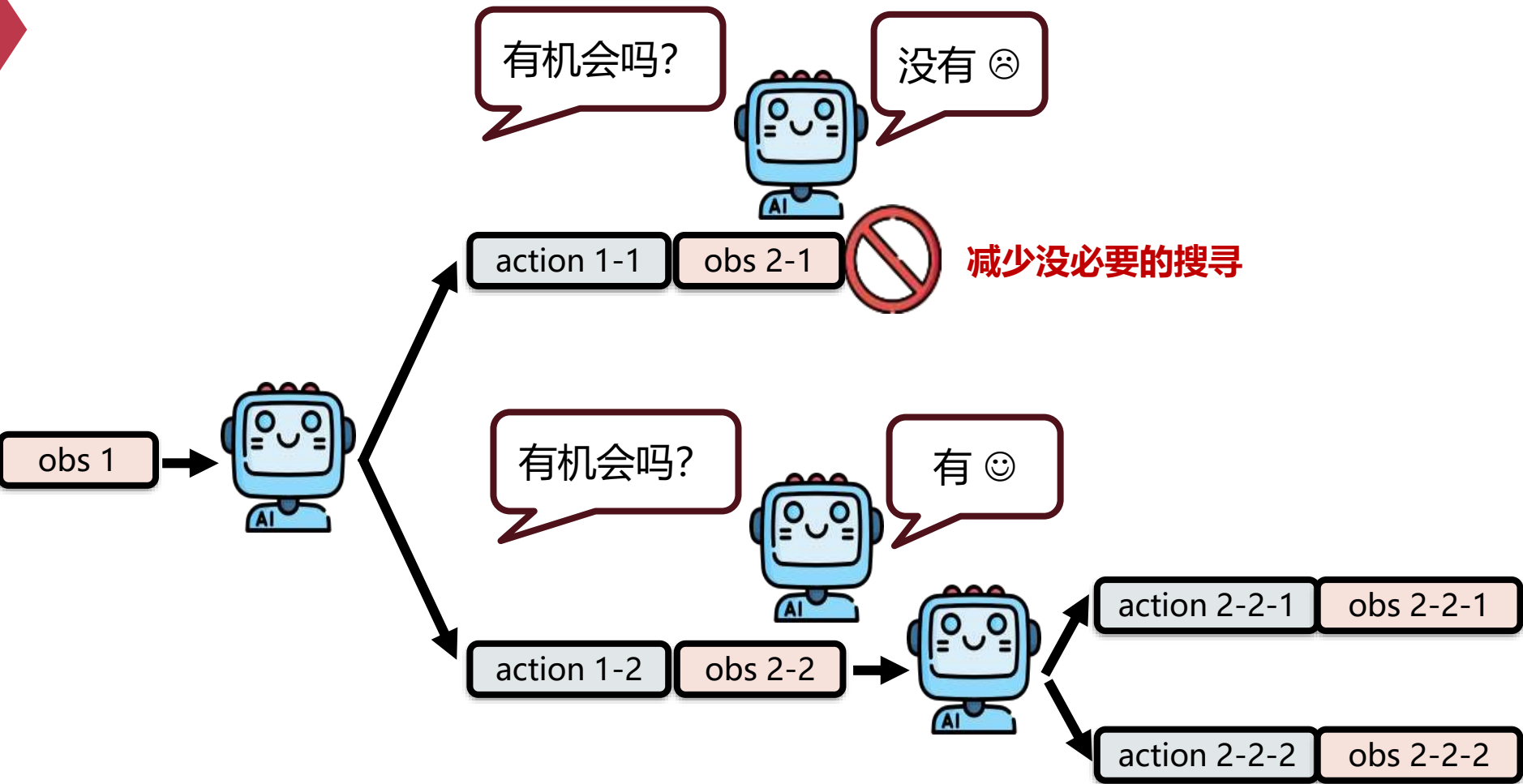
# 智能体规划与 多智能体协作

# 智能体规划能力

不同路径实际试试看？







普通 LLM agent 往往是：

- 想一步
- 做一步
- 再想一步

### CoT (Chain-of-Thought)

- 一条推理链往下走
- 本质是线性思考

### ReAct

- 推理 + 行动交替进行
- 也是以单路径为主

### ToT (Tree of Thoughts)

- 明确提出“树状思维”
- 同时探索多个思路分支

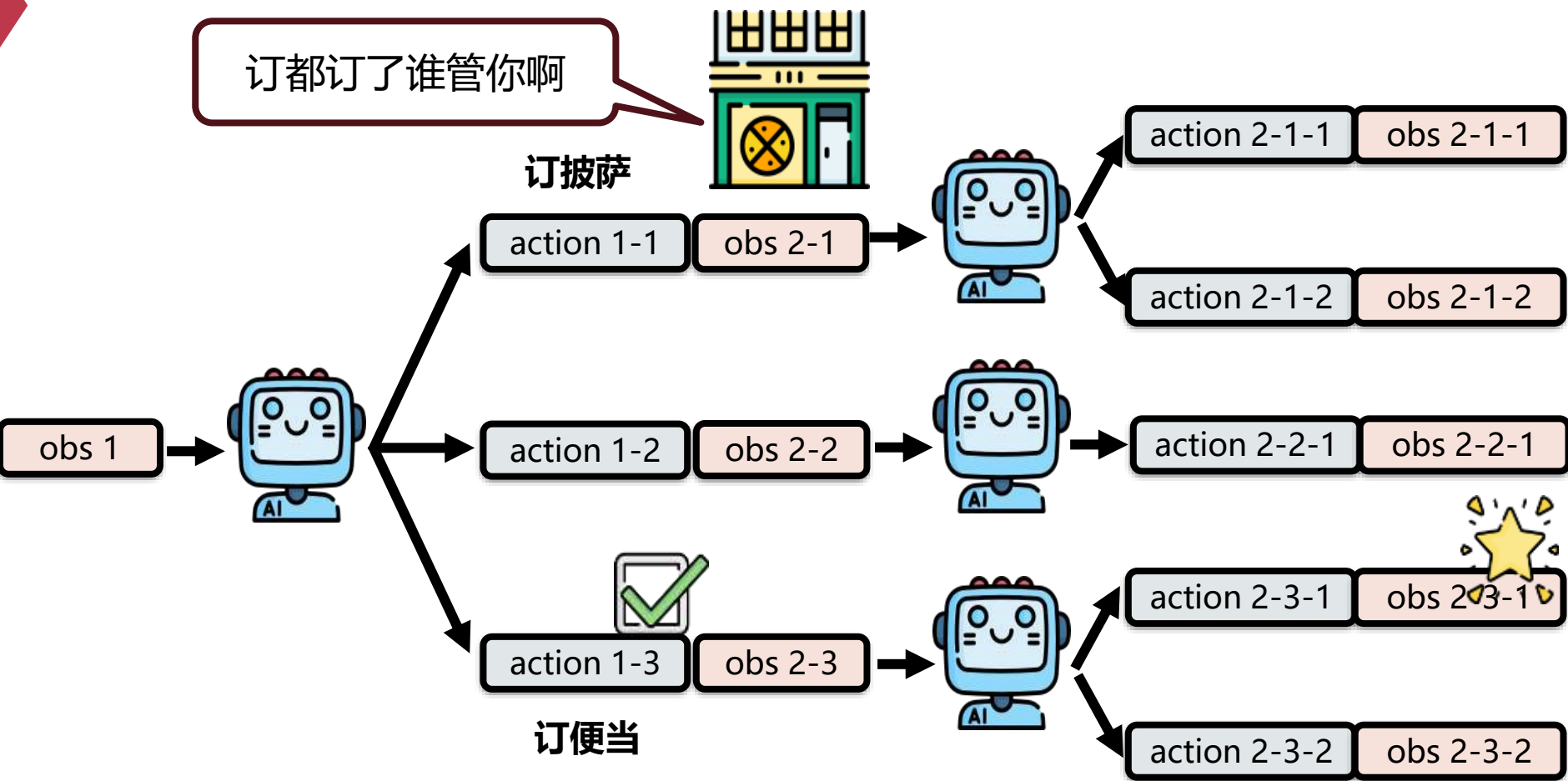
这种方式容易出现：

- 一开始走错方向，后面越错越远
- 局部最优
- 缺少全局规划
- 遇到复杂任务时稳定性差

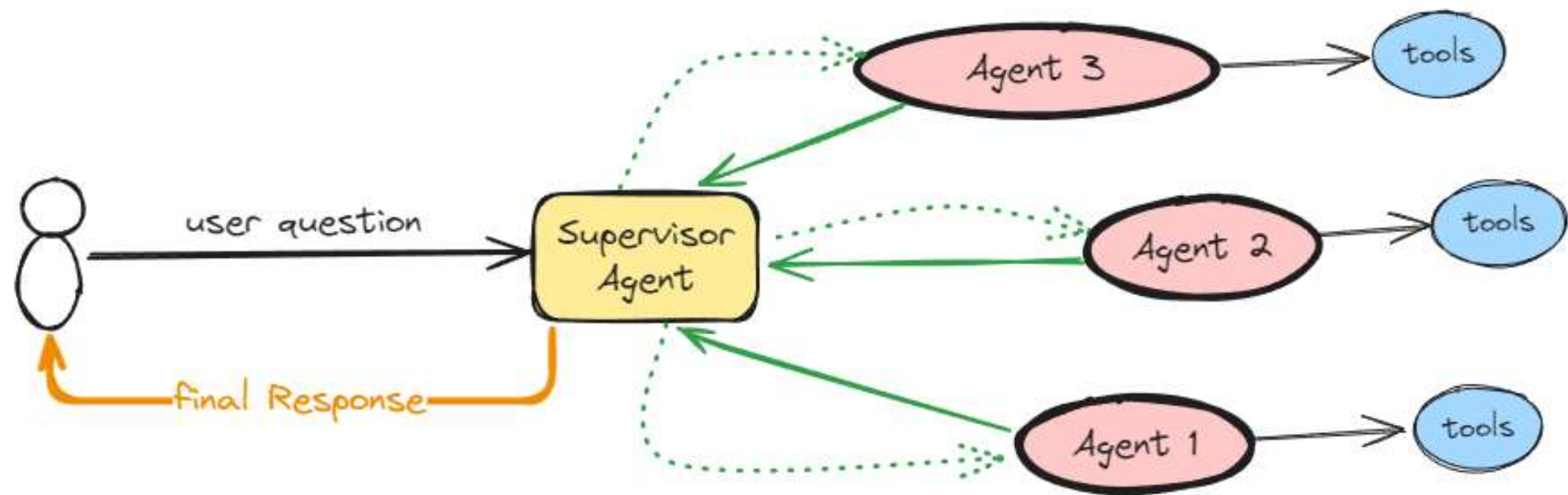
Tree Search 的目标就是：

- 让 agent 不只“顺着往下写”
- 而是先扩展多个候选动作，再评估，再选择

缺点：有些动作无法回溯



# 多智能体协作



# 多智能体协作优势

- **专业分工**：各司其职，效果更精准
- **降低复杂度**：任务拆解，上下文更清晰
- **并行高效**：多任务同时推进，响应更快
- **相互校验**：互审互评，减少幻觉与错误
- **灵活可扩展**：按需增减角色，架构易演进

# 多智能体协作适用场景

- **软件开发**：写码 / 审查 / 测试 / 修复 分工协作
- **深度研究**：多路并行检索 + 汇总核查
- **多视角决策**：辩论、投票、方案评估
- **模拟与仿真**：角色扮演、社会行为模拟

# 智能体工具与CSAPP

- 多进程调度
- 并发与同步
- 进程间通信



# 案例



# Agent Loop 四步闭环

- **1. 感知 (Perception) -- Agent 的感官**
  - 接收用户输入 + 解析系统提示词 + 从记忆加载上下文
- **2. 认知 (Cognition) -- Agent 的大脑**
  - LLM 推理：分析状态、制定计划、决定下一步行动
- **3. 行动 (Action) -- Agent 的手**
  - 调用 API、执行代码、查询数据库 (Function Calling)
- **4. 观察 (Observation) -- Agent 的反馈**
  - 接收工具结果 → 追加进上下文 → 判断是否完成
- **未完成 → 返回步骤 2 继续循环**

# 代码实战：最小化 Agent (60 行)

- **核心结构：**

- 1. tools = [...]      # 工具定义 (JSON Schema)
- 2. execute\_tool(name, args) # 工具执行函数
- 3. run\_agent(query)      # Agent Loop 核心

- **关键观察点：**

- messages 列表 = 短期记忆，每次循环后都在变长
- tool\_calls 的存在与否 = 终止判断 (LLM 自主决定)
- execute\_tool = "行动"模块 (真实系统里是 API / 数据库 / 文件)
- max\_iterations = 安全兜底，防止死循环

# 参考致谢

- 李宏毅老师 AI Agent 课程
  - <https://www.youtube.com/watch?v=M2Yg1kwPpts&t=448s>
- Datawhale 《从零开始构建智能体》第 1 章